



## Volume 2 - Commandes de la station sol

Traçage précis de `stgs_ground_station`

Chaque commande est suivie depuis le parsing CLI jusqu'au résumé final. Les différences TCP, UDP, replay et auto-port sont explicitées, ainsi que les options de pipeline, santé, filtrage et sortie.

Space Telemetry Ground Station (STGS)

Documentation basée sur l'archive `cpp-space-telemetry-ground-station-showcase-modular.zip`.

Les numéros de lignes correspondent à cette révision et peuvent évoluer après modification du code.

## Table des matières

|           |                                              |          |
|-----------|----------------------------------------------|----------|
| <b>1</b>  | <b>Point d'entrée commun</b>                 | <b>2</b> |
| <b>2</b>  | <b>Commande TCP classique</b>                | <b>2</b> |
| 2.1       | Parsing exact . . . . .                      | 2        |
| 2.2       | Chaîne d'appels . . . . .                    | 2        |
| 2.3       | Pourquoi chaque étape existe . . . . .       | 2        |
| <b>3</b>  | <b>Commande UDP</b>                          | <b>3</b> |
| <b>4</b>  | <b>Sélection automatique d'un port local</b> | <b>3</b> |
| <b>5</b>  | <b>Replay STGF</b>                           | <b>3</b> |
| <b>6</b>  | <b>Sortie CSV et JSON</b>                    | <b>4</b> |
| 6.1       | CSV implicite . . . . .                      | 4        |
| 6.2       | JSON implicite . . . . .                     | 4        |
| 6.3       | Forçage du format . . . . .                  | 4        |
| <b>7</b>  | <b>Filtrage des signaux reçus</b>            | <b>4</b> |
| <b>8</b>  | <b>Santé NOMINAL / DEGRADED</b>              | <b>4</b> |
| <b>9</b>  | <b>Logging, verbose et couleurs</b>          | <b>5</b> |
| <b>10</b> | <b>Table complète des bornes station</b>     | <b>5</b> |

## 1 Point d'entrée commun

Toutes les commandes de station commencent par :

### Appels communs

```
main(argc, argv) → parseArgs(argc, argv) → run(Options) → TerminalUi
→ Logger + StationHealthMonitor + PosixSignalStopController →
GroundStationPipeline::run()
```

**Source :** apps/ground\_station.cpp, lignes 15–27 ; **Source :** apps/ground\_station/GroundStationApp.cpp, lignes 107–124

## 2 Commande TCP classique

```
./build/stgs_ground_station --tcp --port 9000 --decoder-threads 4 --output telemetry.csv
```

### 2.1 Parsing exact

`parseArgs()` sélectionne `Transport::Tcp`, valide le port dans `[1,65535]`, fixe 4 workers et le chemin de sortie. Le format est `Auto` ; l'extension n'étant pas `.json`, `resolveOutputFormat()` choisira `CSV`.

### 2.2 Chaîne d'appels

#### TCP sur le port 9000

```
main → parseArgs → run → GroundStationPipeline ctor → TelemetryOutput ctor
→ pipeline.run → startWorkers → receiveNetwork → NetworkServer::run →
NetworkServer::runTcp → createBoundSocket → bind + listen + poll → accept
→ recv → StreamFrameExtractor::feed → callback → submit → rawQueue →
decoderLoop/decodeFrame → decodedQueue → writerLoop → santé + STGA + CSV
```

### 2.3 Pourquoi chaque étape existe

| Fonction                                  | Action                                        | Pourquoi                                                          |
|-------------------------------------------|-----------------------------------------------|-------------------------------------------------------------------|
| <code>createBoundSocket()</code>          | socket IPv4, <code>SO_REUSEADDR</code> , bind | Centraliser l'initialisation et garantir le RAII en cas d'erreur. |
| <code>runTcp()</code>                     | non-blocking + listen + poll                  | Supporter plusieurs clients et un arrêt coopératif.               |
| <code>StreamFrameExtractor::feed()</code> | tranche les frontières                        | TCP ne préserve pas les messages.                                 |
| <code>submit()</code>                     | attribue une séquence + push                  | Stabiliser l'ordre avant parallélisme.                            |
| <code>decodeFrame()</code>                | valide protocole et CRC                       | Distinguer corruption de transport et trame valide.               |
| <code>writerLoop()</code>                 | remet dans l'ordre                            | Garantir CSV/santé identiques avec 1 ou N workers.                |
| <code>processApplicationPayload()</code>  | tranche STGA                                  | Garder la compatibilité avec les payloads opaques.                |
| <code>TelemetryOutput::write()</code>     | écrit CSV/JSON                                | Isoler les effets de bord disque du décodage parallèle.           |

#### Arrêt

Ctrl+C ou SIGTERM bascule `running=false`. Avec le timeout réseau par défaut, la boucle `poll()` réévalue l'arrêt au plus tard après environ 250 ms, hors temps de drainage en cours.

### 3 Commande UDP

```
./build/stgs_ground_station --udp --port 9000 --decoder-threads 4 --output telemetry.csv
```

Le début du pipeline est identique, mais `NetworkServer::run()` appelle `runUdp()`. Chaque datagramme est transmis comme un candidat complet. `MSG_TRUNC` permet de connaître la taille réelle : tout datagramme supérieur à 4123 octets est rejeté avant le codec.

#### Différence essentielle UDP

```
recvfrom(MSG_TRUNC) → taille <= MaxFrameSize ? → callback(ByteVector) →  
pipeline commun
```

**Source :** `src/NetworkServerUdp.cpp`, lignes 18–77

### 4 Sélection automatique d'un port local

```
./build/stgs_ground_station --tcp --auto-port 9000:9010 --output telemetry.csv
```

`parseArgs()` interdit la combinaison `-port + -auto-port`, exige TCP et limite la plage à 256 ports. Ensuite `run()` appelle `selectAutomaticPort()` avant de construire le logger et le pipeline.

#### Auto-port station

```
selectAutomaticPort → findFirstAvailableLocalPort → checkLocalPortAvailability(port  
9000) → socket + bind temporaire → si occupé : port suivant → premier bind  
réussi = port choisi → fermeture de la socket de test → démarrage normal de la  
station sur ce port
```

#### Ce que `-auto-port` ne fait pas

Il ne cherche pas une communication existante. Il cherche le premier port **libre pour bind** sur la machine locale. La recherche d'un listener existant est la fonction de `stgs_port_check` et de `-discover-ports` côté simulateur.

### 5 Replay STGF

```
./build/stgs_ground_station --replay frames.stgf --replay-rate 100 --output replay.csv
```

`parseArgs()` interdit le mélange replay + TCP/UDP. `receiveInput()` choisit `receiveReplay()`.

#### Replay

```
FrameFileReader(path) → validation magic STGF + version → readNext() →  
longueur u32 vérifiée [27,4123] → lecture exacte de la trame → submit() →  
pipeline normal decode/CRC/santé/STGA/export
```

Avec `-replay-rate 0`, aucune temporisation n'est ajoutée. Au-dessus de 0, le code utilise `steady_clock` et `sleep_until` avec une échéance cumulée ; le temps du décodage n'est donc pas ajouté à chaque période.

**Source :** `apps/ground_station/GroundStationInput.cpp`, lignes 30–63 ; **Source :** `src/Replay.cpp`, lignes 103–147

## 6 Sortie CSV et JSON

### 6.1 CSV implicite

```
./build/stgs_ground_station --tcp --port 9000 --output telemetry.csv
```

L'extension autre que `.json` entraîne CSV en mode Auto.

### 6.2 JSON implicite

```
./build/stgs_ground_station --tcp --port 9000 --output telemetry.json
```

L'extension exactement `.json` entraîne JSON.

### 6.3 Forçage du format

```
./build/stgs_ground_station --tcp --port 9000 --output data.txt --output-format json
```

Le format demandé gagne sur l'extension.

`TelemetryOutput` ouvre le fichier en troncature dès la construction du pipeline. Chaque trame **validée** est exportée ; une trame rejetée CRC/format n'est pas écrite. Une erreur STGA interne ne retire pas la trame externe de l'export, car le framing et le CRC restent valides.

**Source :** `apps/ground_station/TelemetryOutput.cpp`, lignes 23–72

## 7 Filtrage des signaux reçus

```
./build/stgs_ground_station --tcp --port 9000 --signal-filter none
./build/stgs_ground_station --tcp --port 9000 --signal-filter moving-average --filter-
    window 5
./build/stgs_ground_station --tcp --port 9000 --signal-filter sine-projection
```

| Mode                   | Fonction appelée                    | Conséquence                                               |
|------------------------|-------------------------------------|-----------------------------------------------------------|
| <b>none</b>            | copie des samples                   | Affiche le signal brut comme filtré.                      |
| <b>moving-average</b>  | <code>movingAverageFilter()</code>  | FIR centré, fenêtre impaire.                              |
| <b>sine-projection</b> | <code>sineProjectionFilter()</code> | Estime DC + coefficients sin/cos à la fréquence nominale. |

Après filtrage, `computeSignalMetrics()` calcule RMS brut, RMS filtré, RMS du résidu, amplitude estimée et SNR de démonstration. Le résultat filtré est rendu dans le terminal ; le CSV/JSON conserve la trame reçue et son payload brut.

## 8 Santé NOMINAL / DEGRADED

Options principales :

```
--degraded-window 100
--degraded-min-samples 20
--degraded-rejection-rate 0.10
--degraded-recovery-rate 0.03
--degraded-critical-count 8
--recovery-critical-count 3
```

Une trame CRC/format rejetée appelle `recordRejected()`. Une trame valide appelle `recordDecoded()` ; CRITICAL et SAFE\_MODE comptent comme critiques. Aucune transition n'est évaluée avant `minSamples`.

#### Hystérésis par défaut

NOMINAL  $\xrightarrow{\text{rejets} \geq 10\% \text{ OU critiques} \geq 8}$  DEGRADED  $\xrightarrow{\text{rejets} \leq 3\% \text{ ET critiques} \leq 3}$  NOMINAL

**Source :** `src/StationHealth.cpp`, lignes 70–110

## 9 Logging, verbose et couleurs

```
./build/stgs_ground_station --tcp --port 9000 --log station.log --log-level info
./build/stgs_ground_station --tcp --port 9000 --verbose
./build/stgs_ground_station --tcp --port 9000 --no-color
```

`-verbose` active aussi le niveau Debug et journalise chaque trame décodée avec sa séquence. `-no-color` n'empêche pas la structure de l'UI ; il supprime seulement les codes ANSI.

## 10 Table complète des bornes station

| Option                        | Borne            | Justification                                         |
|-------------------------------|------------------|-------------------------------------------------------|
| <code>-port</code>            | 1..65535         | Port 0 refusé à l'utilisateur.                        |
| <code>-auto-port</code>       | $\leq 256$ ports | Diagnostic local volontairement borné.                |
| <code>-decoder-threads</code> | 1..256           | Garde-fou CLI, pas limite protocole.                  |
| <code>-queue-capacity</code>  | 1..1 000 000     | Évite zéro et saisies démesurées.                     |
| <code>-replay-rate</code>     | $\geq 0$ , fini  | 0 = débit maximal.                                    |
| <code>-filter-window</code>   | impair, 1..1017  | Filtre centré ; borne dérivée du bloc signal maximal. |
| Fenêtre santé                 | 1..1 000 000     | Garde-fou d'allocation.                               |
| Probabilités santé            | [0,1]            | Taux mathématiquement valide.                         |